Web Cheat sheet

feel free to ask any question

program.cs

```
builder.Services.AddDbContext<DbContext, ExamenContext>();
builder.Services.AddScoped<Type>(p => typeof(GenericRepository<>));
builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();

// creation des instances des services
builder.Services.AddScoped<IServiceEntity, ServiceEntity>();
```

Steps :
1 create controller EntitynameController
2 inject services in controller
IServicEntity serviceEntity; , generate constructor
3 in action name , right click , generate view, razor view, select the right view template (list for index crate for create ….)
select the right entity name
4/add view

search by : date, type ,enum

1 search by int /String

index view

```
<form asp-action="index">
    <fieldset>
        <legend> Recherche</legend>
        Saisir une capacité :
        <input type="int/string" name="name" />
        <input type="submit" value="Serach" />
    </fieldset>
</form>
```

Get Controller

```
 // GET: PlaneController
       public ActionResult Index(int/string? name)
       {
          if(name==null)
             return View(SP.GetAll());
          else
          return View(SP.GetMany(p=>p.entity filed > name));
         //  return View(SP.GetMany(p=>p.entity filed .equals( name)));

       }
```

Seach by date

index view

```
<form asp-action="index">
<fieldset>
<legend> Recherche</legend>
Saisir une date de départ : <input type="date" name="name" />
<input type="submit" value="Serach" />
</fieldset>
</form>
```

Get Controller

```
 // GET: FlightController
        public ActionResult Index(DateTime? name)
        {
            if (name == null)
                return View(sf.GetAll());
            else
                return View(sf.GetMany(f =>
f.FlightDate.Date.Equals(dateDepart)));
        }
```

Search by enum

index View

```
<form asp-action="index">
<fieldset>
<legend> Recherche</legend>
Saisir un type:
  <select
asp-items="Html.GetEnumSelectList<E.ApplicationCore.Domain.enumname>
()"   name="type" >
</select>
<input type="submit" value="Serach" />
</fieldset>
```

Get Controller

```
 public ActionResult Index(enumname? type)
       {
          if(type==null)
          return View(serviceCompte.GetAll());
          return View(serviceCompte.GetMany(p => p.Type.Equals(type)));

       }
```

Create with static enum or dynamic List:

static enum :

in create View :

change the simple input with :

```
 <select asp-for="Type" class="form-control"
asp-items="Html.GetEnumSelectList<E.ApplicationCore.Domain.enumName>
()"
              ></select>
```

enumName: enum in database

dynamic List

step 1

config in controller (get create) : creation of view bag

```
 // GET: CompteController/Create
       public ActionResult Create()
       {
          ViewBag.viewbagname = new SelectList(serviceBanque.GetAll(),
"primary key", "display prop");
          return View();
       }
```

// new SelectList(source , “primary key “, “display prop”)

step2

1/generate view

2/ change simple input to :

```
    <select asp-for="entityFK" class="form-control" asp-items="
ViewBag.viewbagname"
             ></select>
```

Controller example with all crud + Views

in order to test the edit/delete/details, go to the bottom of the index file (action links)and remove the comment on ‘id=item.PrimaryKey’ then replace PrimaryKey by the actual primary key of the entity as shown below :

```
 @Html.ActionLink("Edit", "Edit", new {  id=item.EntityPrimarykey  }) |
 @Html.ActionLink("Details", "Details", new {  id=item.PrimaryKey  }) |
```

```
@Html.ActionLink("Delete", "Delete", new {  id=item.PrimaryKey })
```

```
using E.ApplicationCore.Domain;
using E.ApplicationCore.Interfaces;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;

namespace E.Web.Controllers{
   public class EntityController : Controller    {
      IServiceEntity serviceEntity;
      public CompteController(IServiceEntity serviceEntity,)       {
         this.serviceEntity = serviceEntity
      }
    // GET: CompteController index with search
        public ActionResult Index(TypeCompte? type)
      {
         if(type==null)
         return View(serviceCompte.GetAll());
         return View(serviceCompte.GetMany(p => p.Type.Equals(type)));
      }
// GET: CompteController index without search
        public ActionResult Index()       {
                return View(serviceCompte.GetAll());
      }
```

either with search or without, you can”t have both / if u have multiple search , just add a second one in the same index

```
// GET: CompteController index with Order
        public ActionResult Index()       {
 return View(serviceCompte.GetAll().OrderBy(f=>f.order criteria);  OR
OrderByDescending

);
```

```
}

 // GET: CompteController/Details/5
public ActionResult Details(String id)
{
    return View(serviceCompte.GetById(id));

}

// GET: CompteController/Create
public ActionResult Create()
{
```
if u have a Dynamic list in the create, add the view bag here, as mentioned above(dynamic list section) otherwise, leave it empty
```
    return View();
}



 // POST: CompteController/Create
[HttpPost]
[ValidateAntiForgeryToken]
public ActionResult Create(Compte c)
{
    try
    {
        serviceCompte.Add(c);
        serviceCompte.Commit();
        return RedirectToAction(nameof(Index));
    }
    catch
    {
        return View();
    }
}

// GET: CompteController/Edit/5 Get edit
public ActionResult Edit(String id)
{
```

```
            ViewBag.BanqueList = new SelectList(serviceBanque.GetAll(),
"Code", "Nom"); // only if  u have a  Dynamic list , otherwise leave it
empty

            return View(serviceCompte.GetById(id));
        }

          // POST: CompteController/Edit/5
        [HttpPost]
        [ValidateAntiForgeryToken]
        public ActionResult Edit(Type of primary key  id, Compte c1)
        {
            try
            {

                serviceCompte.Update(c1);
                serviceCompte.Commit();
                return RedirectToAction(nameof(Index));
            }
            catch
            {              return View();           }       }




    // GET: CompteController/Delete/5
        public ActionResult Delete(int id)
        {
            return View();
        }

          // POST: CompteController/Delete/5
        [HttpPost]
        [ValidateAntiForgeryToken]
        public ActionResult Delete(Type of primary key id, IFormCollection
collection)
        {
```

```
        try
        {
            serviceCompte.Delete(serviceCompte.GetById(id));
            serviceCompte.Commit();

            return RedirectToAction(nameof(Index));
        }
        catch
        {
            return View();
        }
    }
}
```

call external controller's method:

at bottom of index file

@Html.ActionLink(linkText: "**displayname**", actionName: "**methode** ", controllerName: "**external controlle**", routeValues: new { id = item.**idFK** }, htmlAttributes: null) |

—----------

something that u don't actually need

Shared ->Layout

```
  <li class="nav-item">
 <a class="nav-link text-dark" asp-area="" asp-controller="Plane"
asp-action="Index">plane index</a>  </li>
```